



LECTURE 9

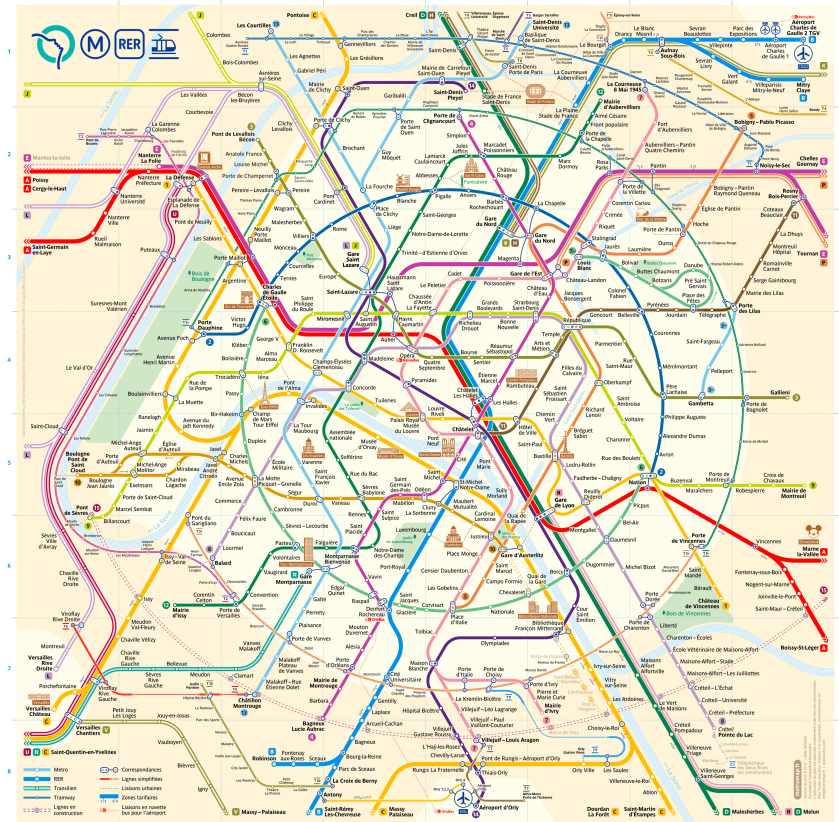
DATALOG: LET'S GET *LOGICAL*

DATASCI290: NEUROSymbolic AI

FEBRUARY 4 2026

TRAIN LINES (LINKS)

Line (Link)	Station	Next Station
4	St.-Germain	Odeon
4	Odeon	St.-Michel
4	St.-Michel	Chatelet
1	Chatelet	Louvre
1	Louvre	Palais-Royal
1	Palais-Royal	Tuileries
1	Tuileries	Concorde
9	Pont de Sevres	Billancourt
9	Billancourt	Michel-Ange
9	Michel-Ange	Iena
9	Iena	F. D. Roosevelt
9	F. D. Roosevelt	Republique
9	Republique	Voltaire



REASONING OVER GRAPHS

- Many databases *implicitly* contain a **graph**
 - stations connected by links, people connected to ancestors, webpages connected by links,..
- Typical graph questions are about **paths**, not just single edges:
 - What stations are reachable from Odeon?
 - What lines can be reached from Odeon?
 - Can we go from Odeon to Chatelet?
 - Are all pairs of stations connected?
 - Is there a cycle (a station reachable from itself in ≥ 1 stops)?
- These are all versions of *transitive closure*:
 - From “Link(Station, NextStation)” infer “Reachable(Station, OtherStation)” for *any path length*
- **Transitive Closure**
 - Transitive closure takes a “direct link” relation ($A \rightarrow B$) and turns it into a “can reach” relation, where A is related to B if you can get from A to B by following one or more links in a chain.
 - In other words, the transitive closure of a relation is a new relation that contains exactly all pairs (A,B) where B is reachable from A by following the original relation repeatedly one or more times.

LIMITATIONS OF RELATIONAL ALGEBRA/ CALCULUS

- Why classic relational calculus / algebra struggles:
 - We can ask “reachable within $\leq n$ stops” for a fixed n ,
 - But “reachable in any number of stops” needs a way to *repeat a rule until no new answers appear*
 - We need **recursion** !
- Consider some facts
 - patient(child1)
has_age_years(child1, 6)
has_weight_lb(child1, 44)
 - reports(child1, fever), reports(child1, vomiting), reports(child1, abdominal_pain), reports(child1, lethargy)
 - hydration_status(child1, low_urine_output)
 - medication(tylenol), medication(motrin)
has_concentration(tylenol, "160mg/5mL")
 - red_flag(low_urine_output), red_flag(lethargy)
suggests(red_flag(low_urine_output), urgent_eval)
- CareTrace domain questions, are “path questions” !
 - “Given these symptoms + age + urine output, **does any red-flag pathway lead to urgent care / ER tonight?**”
 - “Given weight + concentration, **what dosing recommendation is reachable through the medication guideline path?**”
 - “Which conditions are plausible if fever + vomiting + abdominal pain, and **which advice nodes become active?**”

RECURSION

- Facts (stored data; illustrative CareTrace mini-KG facts)
 - `symptom(child1, fever).`
 - `symptom(child1, vomiting).`
 - `sign(child1, low_urine_output).`
 - `age_years(child1, 6).`
- “Clinical graph” edges we want to traverse (a tiny guideline graph)
 - `triggers(low_urine_output, dehydration_risk).`
 - `triggers(dehydration_risk, urgent_eval).`
 - `triggers(lethargy, urgent_eval).`
- Define reachability over these triggers (this *is* transitive closure)
 - `reach(X, Y) :- triggers(X, Y).`
 - `reach(X, Y) :- triggers(X, Z), reach(Z, Y).`
- Use **closure** to answer a CareTrace decision question (derived conclusion)
 - `needs(child1, urgent_eval) :- sign(child1, S), reach(S, urgent_eval).`
 - Means: *If a sign S is present, and by repeatedly following “triggers” edges you can reach “urgent_eval”, then the child needs urgent evaluation.*
- Why this matters:
 - We don’t have to hardcode “2 steps” or “3 steps” deep.
 - The recursion naturally captures “any number of steps,” just like “any number of metro stops.”

WHAT IS DATALOG

- Datalog is a **rule-based query language** for databases: we state **facts** and **rules**, and the system derives what follows.
- Designed for questions that require **chaining inferences**, especially **graph/path queries** (like reachability / transitive closure).
- Program = two kinds of predicates:
 - EDB (extensional): **stored facts** (base data; e.g., symptom(child1, fever) or link(odeon, st_michel)).
 - IDB (intentional): **derived relations** defined by rules (e.g., reach(X,Y) or needs(P, urgent_eval)).
- Rule shape (Horn-clause style):
 - Head :- Body1, Body2, ...
 - Read :- as “**if**”; commas mean **AND**.
- Variables are implicit and universally quantified:
 - Capitalized symbols like X, Y, P are variables; constants are specific entities (stations, patients, symptoms).
- Querying means: “Which bindings make this predicate true?”
 - Example: ask reach(odeon, Y) to list all reachable stations Y.
- Key capability vs. basic relational calculus/algebra:
 - Datalog supports **recursion**, letting rules apply repeatedly to capture “any number of steps.”
- Execution intuition:
 - Start with the facts, apply rules to derive new facts,
 - repeat until no new facts are produced (**fixpoint / closure**).
- Output is still **relational** (tables of answers), but computed via logical inference rather than explicit loops.

LOGIC RECAP

- **Predicate:** a relation name applied to arguments
 - `edge(a,b)`, `symptom(child1, fever)`
- **Term:** what fills an argument position
 - **constant:** `a`, `odeon`, `fever`
 - **variable:** `X`, `Y`
- **Atom:** a predicate applied to terms (no “not”)
 - `edge(Odeon, StMichel)`, `symptom(P, fever)`
- **Literal:** an atom **or its negation**
 - positive literal: `edge(a,b)`
 - negated literal: `¬edge(a,b)` (often written `not edge(a,b)` in Datalog-like syntax)
- **Conjunction (AND):** all must be true
 - $A \wedge B$ (Datalog body: `A, B`)
- **Disjunction (OR):** at least one true
 - $A \vee B \vee C$ (this is the standard “clause” form in logic)

SYNTAX

- **Rule form** (Implications):
 - Head :- Body1, Body2, ...
 - meaning: $(\text{Body1} \wedge \text{Body2} \wedge \dots) \rightarrow \text{Head}$
 - Example: $\text{reach}(X,Y) \text{ :- } \text{edge}(X,Z), \text{reach}(Z,Y).$
- **Clause form** (same statement written as a disjunction of literals):
 - $(\text{Body1} \wedge \text{Body2} \rightarrow \text{Head})$ becomes $\neg \text{Body1} \vee \neg \text{Body2} \vee \dots \vee \text{Head}$
 - Example: $\text{edge}(X,Z) \wedge \text{reach}(Z,Y) \rightarrow \text{reach}(X,Y)$
becomes $\neg \text{edge}(X,Z) \vee \neg \text{reach}(Z,Y) \vee \text{reach}(X,Y)$
- **Horn clause** (definition):
 - A clause with **at most one positive (unnegated) literal**
 - Examples:
 - $\neg A \vee \neg B \vee C$  Horn (one positive: C)
 - $\neg A \vee B \vee C$  not Horn (two positives: B, C)
- **Two special Horn cases that matter in Datalog**
 - **Fact:** one positive literal, no negatives (a rule with empty body)
 - $\text{edge}(a,b).$ (clause form is just $\text{edge}(a,b)$)
 - **Constraint / denial:** no positive literals (sometimes used in extensions)
 - $\neg A \vee \neg B$ (equivalently: $A \wedge B \rightarrow \text{false}$)
- Why Horn clauses are “nice” for Datalog:
 - They align with “derive Head from Body”
 - Inference can be done by repeatedly applying rules until no new facts appear (**fixpoint**) — perfect for reachability / transitive closure.

DATALOG: WHY IT MATTERS NOW

- Essentially, Datalog, provides the explicit logic, predictable semantics, and verifiable inference over structured knowledge (for Neurosymbolic AI Agents)
- **Support for Trustworthy AI Agents**
 - **Consistency** → “same facts, same outcome”
 - Datalog turns a case into derived conclusions via explicit rules; phrasing differences don’t change the logical closure.
 - **Reliability** → “default-safe reasoning pathways”
 - Encode escalation pathways (e.g., dehydration red flags → urgent evaluation) so the agent reliably follows them even when the conversation is messy.
 - **Hard boundaries** → “non-negotiables are explicit”
 - Guardrails like dosing limits, contraindications, and “do-not-reassure” conditions live as enforceable rules, not suggestions.
 - **Transparency** → “because...” chains are real
 - Conclusions come with a derivation trail: observed facts → intermediate clinical concepts → recommended action (not post-hoc storytelling).
 - **Auditability** → “test the logic like software”
 - We can unit-test rule sets on scenarios (“low urine output + lethargy must escalate”), review changes, and version-control them.
 - **Trust through neurosymbolic division of labor**

DATALOG SYNTAX: WHAT A PROGRAM IS MADE OF

- A **Datalog program** is just:
 - a set of **facts** (ground statements that are taken as true), and
 - a set of **rules** (templates for deriving new facts from existing ones)
- Datalog works with **predicates** (relations) like `symptom(P, S)` or `risk(P, R)`
 - the arguments are **constants** (e.g., `child1`, `fever`) or **variables** (e.g., `P`, `S`)
- Datalog uses two layers of predicates:
 - **EDB (Extensional Database)**: predicates whose facts are *given as input data*
 - **IDB (Intensional Database)**: predicates whose facts are *derived by rules*

EDB: THE “INPUT FACTS” LAYER

- **EDB predicates** represent what the system is told / observes (the case record)
 - They are written as **facts** ending with a period:
 - `symptom(child1, fever).`
 - `symptom(child1, vomiting).`
 - `sign(child1, low_urine_output).`
 - `weight_lb(child1, 44).`
 - `med_conc_mg_per_5ml(tylenol, 160).`
- EDB facts are **not inferred**, they are the starting point.

IDB: THE “DERIVED CONCLUSIONS” LAYER

- **IDB predicates** are not entered directly; they are **defined by rules**
- IDB facts correspond to conclusions like:
 - `risk(child1, dehydration).`
 - `needs_urgent_eval(child1).`
 - `dose_mg(child1, tylenol, 320).`
- IDB facts are the outputs of inference, what Datalog *computes*.

RULES

- Rule format:
 - Head :- Body1, Body2,
- Reading:
 - :- means “if”
 - commas mean **AND**
- Examples:
 - `risk(P, dehydration) :- sign(P, low_urine_output).`
 - `needs_urgent_eval(P) :- risk(P, dehydration), symptom(P, lethargy).`
- Variables (P) stand for “for all patients P ...”

QUERIES

- A **query** asks for all facts that can be derived for a predicate
- For instance:
 - “What risks are present?” $\rightarrow$ risk(child1, R)?
 - “Does this case require urgent evaluation?” $\rightarrow$ needs_urgent_eval(child1)?
 - “What dose is recommended?” $\rightarrow$ dose_mg(child1, tylenol, D)?

DEPENDENCY, AND PROPAGATION

- A rule creates a **dependency**: to prove the Head, you must prove the Body
 - `needs_urgent_eval(P)` depends on `risk(P, dehydration)` (and possibly others)
- **Propagation** means derived facts enable more derived facts:
 - *from* `sign(child1, low_urine_output)`
derive `risk(child1, dehydration)`
which can then derive `needs_urgent_eval(child1)`
- This “*derived fact triggers more derived facts*” dynamic is the core engine of rule-based reasoning.

RECURSION, TRANSITIVE CLOSURE

- Some dependencies are **recursive**: a predicate depends on itself (indirectly)
- A tiny “clinical pathway graph” idea:
 - `triggers(low_urine_output, dehydration_risk).`
 - `triggers(dehydration_risk, urgent_eval).`
- Define reachability (closure) over triggers:
 - `reach(X,Y) :- triggers(X,Y).`
 - `reach(X,Y) :- triggers(X,Z), reach(Z,Y).`
- Now we can write:
 - `needs_urgent_eval(P) :- sign(P,S), reach(S, urgent_eval).`
- Meaning: if any observed sign can reach “urgent_eval” through the guideline pathway, escalate.

DATALOG DOES NOT ALLOW FUNCTION SYMBOLS !

- Datalog typically allows **constants and variables**, but **not function symbols**
 - disallowed style: `parent(father(X), X)` or `dose_for(weight(X))`
- Why this restriction exists (practical consequence):
 - With function symbols, you can create infinitely many new terms (like `father(father(father(...)))`)
 - Without them, the set of possible ground facts stays **finite**, so evaluation terminates
- We want rule execution to finish and be predictable (no “infinite term-building” while reasoning)

HERBRAND UNIVERSE: THE “VOCABULARY OF THINGS” THE PROGRAM CAN TALK ABOUT

- The **Herbrand universe** is the set of all ground terms that can appear
- In function-free Datalog, it is simply:
 - all constants mentioned in facts/rules
- CareTrace example:
 - constants might include: child1, fever, vomiting, low_urine_output, dehydration, urgent_eval, tylenol
- Practically, the program reasons *only* over these named objects

HERBRAND BASE: ALL POSSIBLE GROUND ATOMS OVER THAT VOCABULARY

- The **Herbrand base** is:
 - all atoms you could form using the predicates in the program and constants in the Herbrand universe
- CareTrace intuition:
 - it includes candidates like `risk(child1, dehydration)`, `needs_urgent_eval(child1)`, etc.
 - most of these are not true; they're just possibilities
- Why we care:
 - a Datalog *execution* decides which of these possible atoms are actually true.

INTERPRETATION: A “CANDIDATE WORLD” OF WHICH FACTS ARE TRUE

- An **interpretation** picks a set of ground atoms (from the Herbrand base) to treat as true
- Think of it as a *hypothesized state of the world*:
 - Interpretation I might include `symptom(child1, fever)` and `risk(child1, dehydration)`
- A rule is **satisfied** if:
 - whenever the Body atoms are true in I, the Head atom is also true in I
- The goal of Datalog semantics is to identify the “right” interpretation(s) induced by facts + rules.

CONSEQUENCE: WHAT IS LOGICALLY IMPLIED BY FACTS + RULES

- A fact is a **logical consequence** if it must be true in every interpretation that satisfies the program
 - If the program plus observations forces `risk(child1, dehydration)`, then it's not a guess - it's a consequence.
- This is the foundation for “trust”: derived conclusions are *entailed by explicit rules*, not improvised.

FIXPOINT SEMANTICS: HOW DATALOG ACTUALLY COMPUTES CONCLUSIONS

- Start with the EDB facts you already have (the observed case data)
- Apply rules to derive new facts (IDB)
- Add newly derived facts to what you know, and repeat
- Stop when a full pass produces **no new facts**
 - that stable set is the **fixpoint** (also called the closure)
- CareTrace domain:
 - once `risk(child1, dehydration)` has been derived, it can trigger `needs_urgent_eval(child1)`
 - the process continues until all such implications are exhausted.

THE IMPORTANCE OF THE FIXPOINT

- Fixpoint evaluation ensures:
 - **completeness** for the given rule set (we don't miss a chain of implications)
 - **stability** (results don't keep changing)
 - **predictable stopping** (key for real-time decision support)
- In domains like CareTrace we
 - Need all relevant “red-flag → escalation” chains to be found
 - Want the reasoning to terminate quickly and reliably

FIXPOINT SEMANTICS ...

```
% start dosing now (t0)
dose_time(P, t0) :- fever(P).

% every scheduled dose produces another one 6 hours later
dose_time(P, plus6h(T)) :- dose_time(P, T).
```

- Is there a fixpoint here ?

FIXPOINT SEMANTICS, NOW ?

```
% Start dosing at the current time T0
dose_time(P, T0) :- fever(P), now(T0).

% Keep scheduling the next dose 6 hours later,
% but only while we're within 72 hours OR until fever-free for 24 hours
dose_time(P, Tnext) :-
    dose_time(P, T),
    Tnext = T + 6h,
    (Tnext < T0 + 72h),
    not_fever_free_24h(P).
```

NEGATION, IN RULES

- **Positive literal:** `has_symptom(P, fever)`
- **Negated literal:** $\neg$ `has_symptom(P, fever)`
 - read as: “we cannot derive that P has fever” (under the usual database-style assumption)
- In rule form, negation almost always appears in the **body**:
 - `eligible_for_home_care(P) :- has_fever(P),  $\neg$  red_flag(P).`
- Why negation matters:
 - it lets us express **exclusions**, **contraindications**, and **default reasoning** (“if no evidence of X, proceed with Y”).

NEGATION-AS-FAILURE AND THE CLOSED-WORLD ASSUMPTION

- In most database settings, Datalog negation is interpreted as **negation-as-failure**:
 - not A succeeds when A cannot be proven from the current facts + rules
- This matches the **closed-world assumption** common in data systems:
 - what's not in the database (and not derivable) is treated as false for decision-making
- Practical modeling:
 - Use negation for *structured absence* (“no known allergy recorded”, “no derived red flag”), not for vague “unknowns.”
 - If “unknown” is important, represent it explicitly (e.g., `unknown_allergy_status(P)`).

STRATIFIED NEGATION: THE “SAFE” WAY TO USE NOT

- If rules can create cycles through negation, semantics can get tricky:
 - problematic pattern (informally): A depends on not B and B depends on not A
- **Stratified negation** avoids that:
 - you can order predicates into layers (strata) so that
 - negation only points **from higher layers to lower layers**
- Intuition:
 - first compute all “positive/recursive” derived facts in lower layers
 - then apply rules with not ... in higher layers
- Stratification: keeps semantics **predictable**, like “compute closure, then filter.”

NEGATION + SAFETY: “RANGE RESTRICTION” IN RULES

- A common rule-of-thumb restriction: every variable in a negated literal must be **bound** by a positive literal in the body
 - *good*: $\text{ok}(P) \text{ :- patient}(P), \neg \text{red_flag}(P).$
 - *bad*: $\text{ok}(P) \text{ :- not red_flag}(P).$ (P ranges over “everything” !)
- Ensures queries stay **domain-bounded** (finite) and evaluation stays well-defined
- Basically negation is for filtering *known candidates*
 - not for inventing new ones !

COMPLEXITY REGIMES

- Two standard complexity lenses:
 - **Data complexity:** rules fixed, data grows (the “database” view)
 - **Combined complexity:** both rules and data are input (the “program analysis” view)
- For Datalog (function-free, no negation):
 - **Data complexity is polynomial-time (PTIME)**
 - evaluation is essentially “apply rules to closure” over a finite set of facts
- With recursion:
 - still PTIME in data complexity (recursion is *controlled* by fixpoint over finite domain)

DATALOG SUMMARY

- We introduced **logic/rules** as a new kind of tool: not for writing procedures, but for stating *relationships that must hold*.
- Datalog gives a compact way to define **derived knowledge** from base data:
 - facts (what we know) + rules (how knowledge propagates) → consequences (what follows).
- The big unlock is **recursion / closure**:
 - it captures “follow this chain as far as it goes” (reachability, escalation pathways, dependency propagation).
- The semantics are **stable and inspectable**:
 - results come from a fixpoint (derive-until-stable), and each conclusion has a rule-based rationale trail.
- This is how we make a system that is **consistent, constraint-aware, and explainable** over structured knowledge.

DATALOG $\Leftrightarrow$ MODERN AI

- Datalog provides a **small, explicit logic layer** that can sit between data, models, and actions.
- Common ways it shows up in practice (high-level landscape):
 - **Over a knowledge graph:** derive new relationships/states from stored facts (e.g., symptoms/signs → risk flags → recommended next step).
 - **With LLMs:** the LLM can (a) extract structured facts from text for the logic layer, and/or (b) turn logic-derived conclusions into clear, grounded explanations.
 - **In the loop:** write derived conclusions back into the KG so the rest of the system (UI, analytics, downstream agents) sees the same consistent state.
- What this helps us achieve:
 - **Consistency:** same underlying facts lead to the same conclusions.
 - **Hard constraints:** non-negotiable rules (eligibility, contraindications, escalation) are explicit and enforceable.
 - **Transparency:** conclusions come with a traceable “because...” chain.
 - **Trust at the decision boundary:** learned components can interpret and propose, while explicit logic governs what is ultimately concluded or allowed.

HOW WILL WE “INTEGRATE THIS IN” ?

- **1) Facts come from somewhere (the “case record”)**
 - database / form fields (age, weight)
 - extracted from text by an LLM (“only peed once today” → low_urine_output)
 - retrieved from a mini knowledge graph (symptoms, medication label facts, guideline links)
- **2) Datalog runs as the reasoning step (derive what follows)**
 - derive intermediate states: “dehydration risk”, “red flag present”, “needs escalation”
 - derive safe recommendations: “eligible for home care advice” vs “requires urgent evaluation”
 - derive computed fields: dosing amount and schedule constraints (from weight + concentration rules)
- **3) Outputs go to three places (so the whole system stays consistent)**
 - **UI:** show the derived conclusion and the key reasons (“these observations triggered these rules...”)
 - **KG / patient timeline:** write back derived facts (risk flags, recommended action category) as new nodes/edges
 - **LLM response generator:** use the derived facts to produce a grounded, patient-friendly explanation and next steps
- **4) Optional loop: new info updates the conclusion**
 - if the user adds a new symptom/ sign, it becomes a new fact → rerun rules → updated derived state
- *LLM/KG help assemble clean facts → Datalog derives the stable, constraint-aware conclusions → UI/KG/LLM present and store them coherently.*